

Python 08

Elias Wachmann

2025

Content

1. Lambda functions
2. Curve fitting
3. Interpolation
4. Input / Output (I/O) [BONUS]

Lambda functions

Lambda functions

Lambda functions are small anonymous functions.

They can take any number of arguments, but can only have one expression.

They are useful for short functions that are only used once.

```
1 x = np.linspace(0, 2*np.pi, 100)
2 my_lambda = lambda x: np.sin(x)
3 y = my_lambda(x)
```

Lambda functions inside other functions (closure)

Maybe you want to create functions at runtime. This can be done using lambda functions inside other functions.

```
1 def create_gaussian(mu, sigma):  
2     return lambda x: np.exp(-(x-mu)**2/(2*  
3         sigma**2))  
4 x = np.linspace(-5, 10, 1000)  
5 my_lambda = create_gaussian(0, 1)  
6 y = my_lambda(x)
```

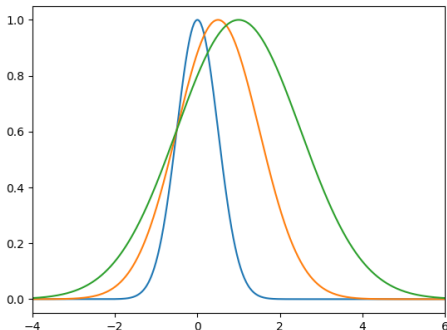
Lambda functions inside other functions (closure)

Create multiple functions using lists and a closure.

```
1 mus, sigmas = [0, 0.5, 1], [0.5, 1, 1.5]
2 ys = [create_gaussian(mu, sigma)(x) for mu,
        sigma in zip(mus, sigmas)]
3 for y in ys:
4     plt.plot(x, y)
5 plt.xlim(-4, 6)
6 plt.show()
```

Lambda functions inside other functions (closure)

Create multiple functions using lists and a closure.



Lambda using multiple arguments

Lambda functions can take multiple arguments, just like normal functions.

```
1 adder = lambda x, y: x + y
2 print(adder(1, 2)) # 3
3 adder_default = lambda x, y=1: x + y
4 print(adder_default(1)) # 2
5 adder_lambda = lambda x: lambda y: x + y
6 print(adder_lambda(1)(2)) # 3
```

You can even specify default values for the arguments and nest lambda functions.

Lambda & if / else

Compact way to find prime numbers.

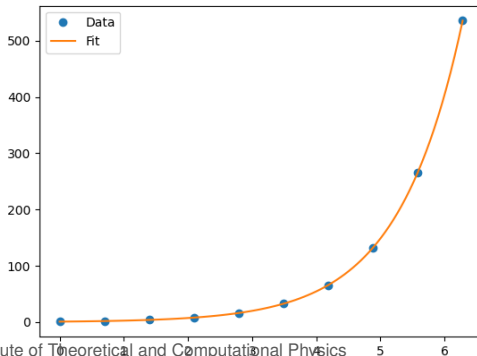
```
1 import numpy as np
2 prime_test = lambda x,y: True if x % y else
   False
3 x = np.arange(1, 101)
4 primes = [i for i in x if all(prime_test(i,j)
   for j in range(2, i))]
5 print(primes[:5]) # [1, 2, 3, 5, 7]
```

Note: The <arg> if <condition> else <arg> syntax is called a conditional expression which can also be used inside list/dict comprehensions and in returns.

Curve fitting

Curve fitting using `scipy`

With the `curve_fit` function from the `scipy` module you can fit a function to data.



Curve fitting using scipy

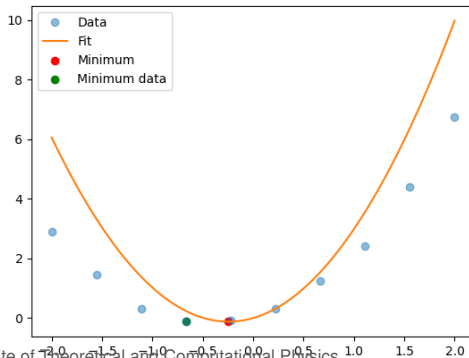
```
1 from scipy.optimize import curve_fit
2
3 x_samples = np.linspace(0, 2 * np.pi, 10)
4 sample_data = np.exp(x_samples)
5 sample_data += np.random.normal(0, 0.1, 10)
6
7 def fit(x, a, b): return a * np.exp(b * x)
8
9 popt, pcov = curve_fit(fit, x_samples,
10                        sample_data)
11 a_guess, b_guess = popt
```

Curve fitting using scipy

```
1 a_guess, b_guess = popt
2
3 x_smooth = np.linspace(0, 2 * np.pi, 100)
4 plt.plot(x_samples, sample_data, 'o', label=
    'Data')
5 plt.plot(x_smooth, fit(x_smooth, a_guess,
    b_guess), label='Fit')
6 plt.legend()
7 plt.show()
```

Finding a minima using scipy

With the `fmin` function you can find the minima of a function.



Finding a minima using scipy

```
1 import matplotlib.pyplot as plt
2 x_samples = np.linspace(-2, 2, 10)
3 sampled_data = np.random.uniform(-2, 2)*
   x_samples**2 + \
4     x_samples + np.random.normal(0, 0.1, 10)
5
6 def sample_data(a, b, x): return a*x**2+b*x
7
8 popt, pcov = curve_fit(sample_data,
   x_samples, sampled_data)
9 a_guess, b_guess = popt
10 fmin_ = fmin(lambda x: sample_data(a_guess,
   b_guess, x), 0)
```

Finding a minima using scipy

```
1 x_smooth = np.linspace(-2, 2, 100)
2 y_smooth = sample_data(a_guess, b_guess,
    x_smooth)
3 plt.plot(x_samples, sampled_data, 'o', label
    = 'Data', alpha=0.5)
4 plt.plot(x_smooth, y_smooth, label='Fit')
5 plt.scatter(fmin_, sample_data(a_guess,
    b_guess, fmin_),
6             c='r', label='Minimum')
7 min_data = min(sampled_data)
```


Finding a minima using scipy

```
1 plt.scatter(x_samples[np.where(sampled_data  
    == min_data)],  
2             min_data, c='g', label='Minimum  
    data')  
3  
4 plt.legend()  
5 plt.show()
```

Interpolation

Difference between interpolation and curve fitting

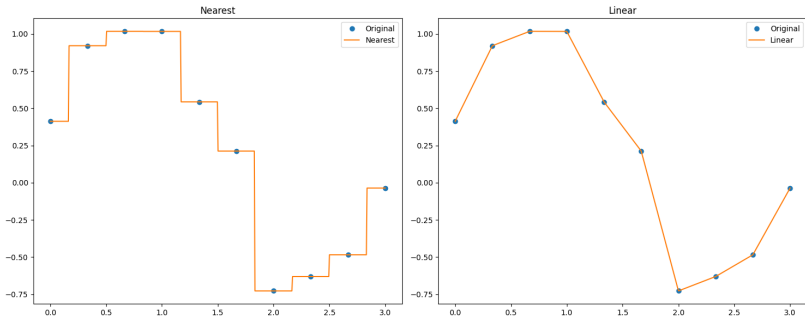
Interpolation constructs new data points within the range of a discrete set of known data points.

Curve fitting is the process of constructing a curve, or mathematical function, that has the best fit to a series of data points.

While interpolation requires the function to **go through the data points**, curve fitting does not.

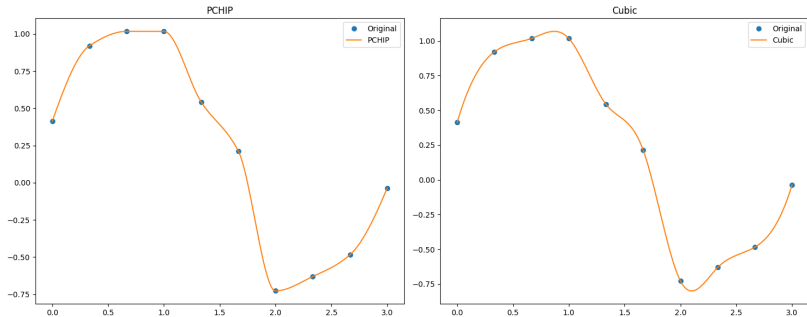
Interpolation using scipy

Scipy has a suite of functions for interpolation.



Interpolation using `scipy`

Scipy has a suite of functions for interpolation.



Interpolation using `scipy`

Using the `interp1d` function you can interpolate data.

- Nearest-neighbor interpolation ('nearest')
- Linear interpolation ('linear')
- Piecewise polynomial interpolation (`PchipInterpolator`)
- Cubic spline interpolation ('cubic')

For piecewise polynomial interpolation use
`PchipInterpolator`.

Interpolation using scipy

```
1 from scipy.interpolate import interp1d,  
   PchipInterpolator  
2  
3 x_data = np.arange(0, 3.1, 1/3)  
4 y_data = np.sin(2*x_data) + np.random.rand(  
   len(x_data))/2  
5  
6 x_int = np.linspace(x_data[0], x_data[-1],  
   500)  
7  
8 interpolation_methods = ['nearest', 'linear',  
   , 'cubic']
```

Interpolation using scipy

```
1 y_interpolated = {}
2 for method in interpolation_methods:
3     f = interp1d(x_data, y_data, kind=method
4                 )
5     y_interpolated[method] = f(x_int)
6 y_nearest = y_interpolated['nearest']
7 y_linear = y_interpolated['linear']
8 y_pchip = PchipInterpolator(x_data, y_data)(
9     x_int)
10 y_spline = y_interpolated['cubic']
11 #... plotting
```


Input / Output (I/O) [BONUS]

Input / Output (I/O)

We have considered the print function to output text to the console.

Input can be read from the console using the input function.

Users can also input arguments when calling a script from the command line. These arguments are stored in the sys.argv list.

Input – `sys.argv`

Let's assume we have a script with the following content:

```
1 import sys
2 def calcprimes(n):
3     return [i for i in range(1, n+1) if all(
4         i % j for j in range(2, i))]
5
6 if __name__ == '__main__':
7     print(calcprimes(int(sys.argv[1])))
```

We can now call this script from the command line and pass the number of iterations as an argument.

Input – `sys.argv`

We can now call this script from the command line and pass the number of iterations as an argument.

```
C:\REPOS\exercises-python>C:/Users/elias/AppData/Local/Programs/Python/Python311/python.exe c:/REPOS/exercises-python/slides/08/examples/argv.py 100  
[1, 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]
```

Entering 100 as an argument will return and print all prime numbers up to 100.

Input – input

The input function can be used to read user input from the console.

```
1 name = input("Name: ")
2 print("Hello, " + name + "!")
3 age = int(input("Age: "))
4 print("You are " + str(age) + " years old.")
```

```
Name: Alice
Hello, Alice!
Age: 42
You are 42 years old.
```

Input – check it before you wreck it!

User input is always read as a string and can be used as it or converted to other types.

Never trust user input!

Always check if the input is valid and convert it to the desired type. Otherwise, it might lead to unexpected behaviour, error or even security issues in the worst case.

Input – check it before you wreck it!

Not checking input leads to unexpected behavior:

```
Name: 42
Hello, 42!
Age: hello
Traceback (most recent call last):
  File "c:\REPOS\exercises-python\slides\08\examples\input.py", line 3, in <module>
    age = int(input("Age: "))
          ^^^^^^^^^^^^^^^^^^^
ValueError: invalid literal for int() with base 10: 'hello'
```

We can fix this by converting to the desired type in a try-except block.

Input – check it before you wreck it!

We can fix this by converting to the desired type in a try-except block.

```
1 name = input("Name: ")
2 print("Hello, " + name + "!")
3 age = input("Age: ")
4 try:
5     age = int(age)
6 except ValueError:
7     print(f"{age} is not a valid age.")
8     quit()
9 print("You are " + str(age) + " years old.")
```


Input – check it before you wreck it!

We can fix this by converting to the desired type in a try-except block.

This way the user gets useful feedback and the program does not crash.

```
Name: Fred  
Hello, Fred!  
Age: AA  
AA is not a valid age.
```

Input – check it before you wreck it!

Some code may be malicious and try to exploit your program.

```
1 size_from_argv = int(sys.argv[1])  
2 large_arr = [0] * size_from_argv  
3  
4 time.sleep(5)
```

If you input a large enough number the program will consume a lot of your RAM and the system may slow down or crash.

Input – check it before you wreck it!

Python has some check which prevent this from happening → `MemoryError`.

```
① etschgi1@etschgi1-ThinkPad-E595 ~/Desktop/REPOS/exercises-python (git)-[slides] % /  
bin/python3 /home/etschgi1/Desktop/REPOS/exercises-python/slides/08/examples/memeater  
.py 1000000000000000000  
Traceback (most recent call last):  
  File "/home/etschgi1/Desktop/REPOS/exercises-python/slides/08/examples/memeater.py"  
, line 5, in <module>  
    large_arr = [0] * size_from_argv  
MemoryError
```

But it is still good practice to check user input and prevent this from happening.

Input – eval

The `eval` function can be used to evaluate a string as a Python expression.

```
1 expr = sys.argv[1]
2 print(eval(expr))
3 print(eval("1 + 2 * 3"))
```

As you should know by now, this is dangerous and you shouldn't evaluate user input! Sometimes it may be useful though to dynamically evaluate equations (just like with closures).

Output – a closer look at print

The print function has a lot of options.

```
1 print("Multiple", "arguments", "to", "print"  
    , sep=" | ", end="!\n")  
2 print("Countdown: ")  
3 for i in reversed(range(10)):  
4     print(i, end="\r")  
5     time.sleep(0.5)
```

The `sep` argument can be used to specify the separator between the arguments and the `end` argument can be used to specify the end of the line.
`\n` ... newline, `\t` ... tabulator, `\r` ... carriage return.

Output – a closer look at colors

How to print colored text to the console?

Use escape sequences!

```
1 print("\033[4mUnderlined text\033[0m\n")
2 print("\033[1;31;40mBright Red Text\033[0m\n")
3 print("\033[91mRed Text\033[0m\n")
4 print("\033[95mBright Magenta Text\033[0m\n")
5 print("\033^Invisible\033[0m\n")
```

Output – a closer look at colors

Output in the console:

Bold text

Underlined text

Bright Red Text

Red Text

Bright Magenta Text

Import csv-files

csv is a module that allows you to read and write csv-files.

csv is a text format, so you have to open the file in text mode (`w` or `r`).

To read a file use the `reader()` method, to write a file use the `writer()` method.

Import csv-files – Example (csv)

```
1 import csv
2 import matplotlib.pyplot as plt
3 lists = []
4 header = []
5 with open('slides/08/examples/example.csv',
6           'r') as f:
7     reader = csv.reader(f)
8     header = reader.__next__()[0].split(';')
9     lists = [[] for _ in range(len(header))]
10    for row in reader:
11        for i, item in enumerate(row[0].
12                                split(';')):
13            lists[i].append(int(item))
14
15 plt.plot(lists[0], lists[1])
16 plt.show()
```

Import csv-files – Example (csv)

Wow, that was a lot of code!
Isn't there a better way?

Maybe do everything in one line?

Use pandas inbuilt csv-import function instead:

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 df = pd.read_csv('slides/08/examples/example
    .csv', sep=';')
4 plt.plot(df['x'], df['y'])
5 plt.show()
```